

RecallStack

Database Tables and API Design Guide

Supabase / PostgreSQL architecture reference

Public application tables	34
Core functional areas	7
Authentication source	Supabase auth.users
Primary design principle	Business-action APIs, not table CRUD

1. The Big Picture

The database is easiest to understand as seven functional areas rather than 34 unrelated tables. The central idea is that users learn stable Content Items, editorial teams manage Content Versions and Blocks, learning state tracks what a user knows, Practice records attempts, Recall schedules revision, and Sync keeps state consistent across devices.

- User
- |
 - +-- learns Content
 - +-- bookmarks Content
 - +-- writes Notes
 - +-- practices Resources
 - +-- reviews Recall Cards
 - +-- syncs personal state across Devices

- Content
- |
 - +-- belongs to a Domain
 - +-- organized by Categories
 - +-- tagged with Topics
 - +-- has multiple Versions
 - +-- each Version contains Blocks
 - +-- published into Catalog Releases

Functional areas

Area	Tables
Identity and authorization	profiles, roles, profile_role_grants
Catalog and taxonomy	domains, categories, topics, topic_categories
Content platform	content_items, content_versions, content_blocks, mappings, relations, workflow history
Learning state	user_progress, bookmarks, user_notes, activity_events
Practice	practice_providers, practice_resources, practice_attempts
Recall	review_cards, review_history
Offline sync and catalog distribution	devices, sync state, mutations, counters, change logs, releases

2. Identity and Authorization

Supabase owns authentication in `auth.users`. RecallStack stores application-specific user information in `public.profiles`. Do not create users directly in the application database; use Supabase Auth and let profiles represent application metadata.

```
auth.users
|
v
profiles
|
+-- user_progress
+-- bookmarks
+-- user_notes
+-- practice_attempts
+-- review_cards
+-- devices
```

profiles

Purpose: Application profile for a Supabase Auth user.

Main data: `display_name`, `avatar_url`, `timezone`, `created_at`, `updated_at`

Key relationships: `profiles.id` references `auth.users.id`. Most user-owned tables reference `profiles.id`.

Typical API surface

Endpoint	Purpose
GET /me	Return current user's profile
PATCH /me	Update display name, avatar metadata, or timezone

Design note: Authentication remains in Supabase Auth. The profile is business data, not credential storage.

roles

Purpose: Master list of authorization roles.

Main data: `code` and `description`, for example `admin`, `content_editor`, `reviewer`.

Key relationships: Referenced by `profile_role_grants`.

Typical API surface

Endpoint	Purpose
GET /admin/roles	List available roles
POST /admin/roles	Create a role, usually admin-only

Design note: This is stable reference data and changes infrequently.

profile_role_grants

Purpose: Historical assignment and revocation of roles.

Main data: profile_id, role_id, granted_at/by, revoked_at/by.

Key relationships: Connects profiles to roles and preserves authorization history.

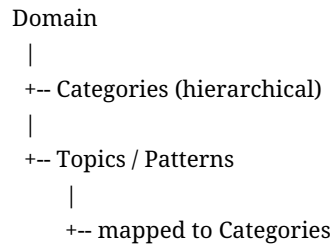
Typical API surface

Endpoint	Purpose
GET /admin/users/{userId}/roles	List active and historical grants
POST /admin/users/{userId}/roles	Grant a role
POST /admin/users/{userId}/roles/{roleId}/revoke	Revoke by setting revoked_at

Design note: Prefer revocation over deleting historical grant rows.

3. Catalog and Taxonomy

Taxonomy provides navigation and conceptual classification. Domains are top-level subject areas; Categories provide a hierarchy; Topics represent reusable concepts or patterns.



domains

Purpose: Top-level subject areas.

Main data: slug, name, description, sort_order, is_active.

Key relationships: Parent scope for categories, topics, content items, catalog releases, and catalog sync.

Examples:
dsa
system-design
machine-learning
database-systems

Typical API surface

Endpoint	Purpose
GET /domains	List active domains
GET /domains/{slug}	Get one domain
POST /admin/domains	Create domain
PATCH /admin/domains/{id}	Edit or deactivate domain

categories

Purpose: Hierarchical navigation tree inside a domain.

Main data: domain_id, parent_category_id, slug, name, sort_order, is_active.

Key relationships: Composite FK ensures a parent category belongs to the same domain.

Typical API surface

Endpoint	Purpose
GET /domains/{domainId}/categories	List top-level or all categories
GET /categories/{id}/children	List child categories
GET /categories/{id}/content	List content assigned to a category

Design note: Use Categories for navigation hierarchy, not as a substitute for Topics.

topics

Purpose: Concepts or patterns that classify content across categories.

Main data: domain_id, kind (topic or pattern), slug, name, description.

Key relationships: Belongs to a domain and maps to categories through topic_categories.

Examples:

Binary Search

Sliding Window

Two Pointers

Dynamic Programming

Breadth First Search

Typical API surface

Endpoint	Purpose
GET /domains/{domainId}/topics	List topics/patterns
GET /topics/{slug}	Get topic details
GET /topics/{id}/content	List content mapped to a topic

topic_categories

Purpose: Many-to-many mapping between Topics and Categories.

Main data: domain_id, topic_id, category_id, sort_order.

Key relationships: Composite FKs enforce same-domain mappings.

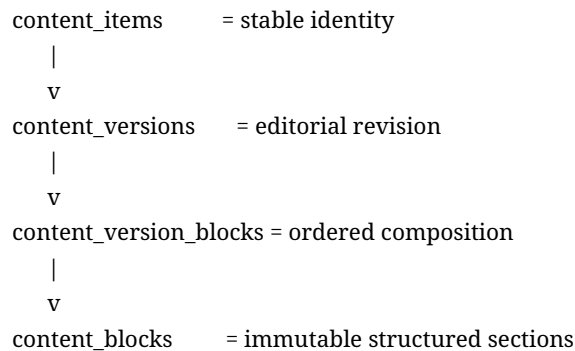
Typical API surface

Endpoint	Purpose
PUT /admin/topics/{id}/categories	Replace or update category mappings

Design note: Usually not exposed as raw table CRUD. Treat it as part of topic administration.

4. Content Platform

This is the center of RecallStack. The model separates stable content identity from editorial versions and from reusable content blocks.



content_items

Purpose: Stable identity for a learning object.

Main data: domain_id, slug, type, difficulty, current_published_version_id, created_by, archived_at.

Key relationships: Belongs to a domain and points to its current published version using a composite ownership FK.

Typical API surface

Endpoint	Purpose
GET /content/{slug}	Return published content document
GET /content?domain=dsa&type=problem	Browse/filter content
POST /admin/content	Create content item
POST /admin/content/{id}/archive	Archive content without losing user history

Design note: Bookmarks, progress, notes, practice history, and recall state attach to content_item_id, not to a specific version.

content_versions

Purpose: Editorial revisions of one Content Item.

Main data: version_number, status, title, summary, search_document, authorship/review metadata, row_version.

Key relationships: Many versions belong to one content item; only one is pointed to as current published version.

Typical API surface

Endpoint	Purpose
GET /admin/content/{contentId}/versions	List versions
POST /admin/content/{contentId}/versions	Create draft version
PATCH /admin/content-versions/{versionId}	Edit draft/in-review version

POST /admin/content-versions/{versionId}/submit-review	Move into review workflow
POST /admin/content-versions/{versionId}/publish	Publish atomically

Design note: Public users should normally resolve the current published version through the Content Item.

content_version_status_history

Purpose: Audit trail of editorial workflow transitions.

Main data: from_status, to_status, changed_by, reason, changed_at.

Key relationships: Belongs to one content version.

Typical API surface

Endpoint	Purpose
GET /admin/content-versions/{id}/history	Show workflow history

Design note: Write this automatically inside workflow transactions; do not expose a generic POST /status-history endpoint.

content_blocks

Purpose: Reusable immutable structured sections of content.

Main data: type, payload JSONB, plain_text, content_hash, created_by.

Key relationships: Referenced by content_version_blocks.

Example block:

```
{
  "type": "invariant",
  "payload": {
    "text": "If the target exists, it remains inside the active search interval."
  }
}
```

Typical API surface

Endpoint	Purpose
POST /admin/content-blocks	Optional low-level editor API
PUT /admin/content-versions/{id}/document	Preferred higher-level document update API

Design note: payload holds structure; plain_text supports indexing/search. Large images and binaries belong in Supabase Storage, not JSONB.

content_version_blocks

Purpose: Ordered composition of blocks within a content version.

Main data: content_version_id, content_block_id, position, heading.

Key relationships: Many blocks form one version; the same immutable block may be reused.

Typical API surface

Endpoint	Purpose
GET /content/{slug}	Backend joins mappings and blocks to build one document

Design note: Usually hidden behind document-level APIs.

content_item_categories

Purpose: Maps content items to navigation categories.

Main data: domain_id, content_item_id, category_id, sort_order.

Key relationships: Composite FKs enforce domain consistency.

Typical API surface

Endpoint	Purpose
PUT /admin/content/{id}/categories	Replace category mappings
GET /categories/{id}/content	Browse content by category

content_item_topics

Purpose: Maps content items to Topics/Patterns.

Main data: domain_id, content_item_id, topic_id, is_primary, sort_order.

Key relationships: Composite FKs enforce domain consistency.

Typical API surface

Endpoint	Purpose
PUT /admin/content/{id}/topics	Replace topic mappings
GET /topics/{id}/content	Browse by topic

Design note: is_primary identifies the dominant concept/pattern for the content item.

content_relations

Purpose: Directed content graph for prerequisites, related items, and alternatives.

Main data: source_content_item_id, target_content_item_id, relation_type, sort_order.

Key relationships: Both ends reference content_items.

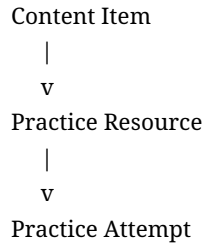
Typical API surface

Endpoint	Purpose
GET /content/{id}/related	Return related and alternative content
GET /content/{id}/prerequisites	Return prerequisite graph edges

Design note: This supports recommendations and learning-path construction.

5. Practice System

Practice is separate from content. Content explains; Practice records what the learner attempted.



practice_providers

Purpose: Registry of internal or external practice sources.

Main data: slug, name, base_url, is_active.

Key relationships: Referenced by practice_resources and practice_attempts.

Examples:
LeetCode
HackerRank
Codeforces
RecallStack Quiz

Typical API surface

Endpoint	Purpose
GET /practice/providers	List active providers
POST /admin/practice/providers	Create provider

practice_resources

Purpose: A concrete question, challenge, quiz, or practice link associated with content.

Main data: content_item_id, provider_id, external_key, url, url_hash, title, is_primary.

Key relationships: Belongs to one content item and one provider.

Typical API surface

Endpoint	Purpose
GET /content/{contentId}/practice-resources	List resources for content
POST /admin/content/{contentId}/practice-resources	Attach resource

Design note: Use is_primary to highlight the recommended resource; archive instead of hard-deleting historically used resources.

practice_attempts

Purpose: Append-only record of a user's practice attempt.

Main data: user_id, content_item_id, practice_resource_id, provider_id, outcome, duration, hint_used, confidence before/after.

Key relationships: Composite resource context guarantees that resource, content item, and provider agree.

Example:

User: Saksham

Content: Sliding Window

Resource: LeetCode 3

Outcome: solved_with_hint

Duration: 1500 seconds

Confidence: 40 -> 70

Typical API surface

Endpoint	Purpose
POST /practice/attempts	Record an attempt
GET /me/practice-attempts	List user's attempts
GET /me/content/{contentId}/attempts	History for one content item

Design note: Normal application behavior should treat attempts as append-only events.

6. Learning State and Activity

These tables describe the user's relationship with content. `user_progress` is current state; `bookmarks` are set membership; `notes` are editable state; `activity_events` are observational events.

user_progress

Purpose: Current learning status for one user-content pair.

Main data: status, confidence, last_opened_at, row_version.

Key relationships: Composite primary key: user_id + content_item_id.

Typical API surface

Endpoint	Purpose
GET /me/progress	List current learning state
GET /me/progress/{contentId}	Get one state row
PUT /me/progress/{contentId}	Create or replace current state

Design note: This is current state, not history. Use row_version for optimistic concurrency where offline edits are possible.

bookmarks

Purpose: Set of content items saved by a user.

Main data: user_id, content_item_id, created_at.

Key relationships: Composite primary key makes membership unique.

Typical API surface

Endpoint	Purpose
GET /me/bookmarks	List bookmarks
PUT /me/bookmarks/{contentId}	Idempotently add bookmark
DELETE /me/bookmarks/{contentId}	Idempotently remove bookmark

Design note: Set semantics make synchronization simpler.

user_notes

Purpose: Private notes, mistakes, and insights attached to content.

Main data: kind, title, body, row_version, deleted_at.

Key relationships: Belongs to one user and one content item.

Typical API surface

Endpoint	Purpose
POST /me/notes	Create note

GET /me/notes	List notes
GET /me/content/{id}/notes	Notes for one content item
PATCH /me/notes/{id}	Edit with row-version check
DELETE /me/notes/{id}	Soft delete for sync tombstone

Design note: row_version and deleted_at make this table suitable for offline conflict handling.

activity_events

Purpose: Observational stream of meaningful user activity.

Main data: event_type, optional content_item_id, optional polymorphic source, metadata JSONB, occurred_at.

Key relationships: Belongs to one user; may reference content.

Possible event types:

content_opened
content_completed
practice_started
practice_finished
review_started
review_completed
bookmark_added

Typical API surface

Endpoint	Purpose
GET /me/activity	Optional timeline endpoint

Design note: Prefer server-generated events from business actions. Avoid letting clients write arbitrary event metadata without validation.

7. Recall and Spaced Revision

The Recall subsystem separates current scheduler state from immutable review events. `review_cards` answers what is due now; `review_history` explains what happened over time.

```
review_cards = current scheduling state
|
v
review_history = append-only review events
```

review_cards

Purpose: Current spaced-repetition scheduling state per user-content pair.

Main data: scheduler name/version, due_at, interval, stability, difficulty, review_count, lapse_count, scheduler_state, row_version.

Key relationships: One card belongs to one user and one content item.

Typical API surface

Endpoint	Purpose
GET /me/reviews/due	Return due unsuspended cards
GET /me/review-cards	List current card states
POST /me/review-cards/{id}/suspend	Suspend card
POST /me/review-cards/{id}/resume	Resume card

Design note: Treat the card as mutable current state. Updates must be part of the same transaction as review-history insertion.

review_history

Purpose: Immutable history of review submissions.

Main data: review_event_id, review_card_id, user_id, rating, response time, previous/next due dates, scheduler state after review.

Key relationships: Composite FK guarantees the review history user owns the referenced card.

Transactional review flow:

1. Deduplicate review_event_id
2. Lock or CAS review_card
3. Calculate next schedule
4. Insert review_history
5. Update review_card
6. Commit

Typical API surface

Endpoint	Purpose
POST /me/reviews/{cardId}/submit	Submit review command atomically
GET /me/reviews/history	Historical review timeline

Design note: Do not expose POST /review-history and PATCH /review-card as separate client operations.

8. Catalog Releases and Distribution

Catalog releases provide stable, versioned snapshots of a domain. They are useful for offline downloads and reproducible content bundles.

catalog_releases

Purpose: Versioned snapshot identity for one domain.

Main data: domain_id, release_number, status, published_at/by, retired_at.

Key relationships: Belongs to one domain and owns a release manifest.

Typical API surface

Endpoint	Purpose
GET /catalog/domains/{domainId}/latest-release	Find latest published release
GET /catalog/releases/{releaseId}	Read release metadata
POST /admin/catalog/releases	Create building release
POST /admin/catalog/releases/{id}/publish	Publish and freeze release

catalog_release_versions

Purpose: Manifest mapping each content item to the exact version included in a release.

Main data: domain_id, catalog_release_id, content_item_id, content_version_id.

Key relationships: Composite FKs enforce release-domain, item-domain, and item-version consistency.

Typical API surface

Endpoint	Purpose
GET /catalog/releases/{releaseId}/manifest	Return exact version manifest

Design note: Usually populated and frozen by the release publication workflow.

9. Devices and Offline Synchronization

The sync subsystem handles device identity, outbound mutations, inbound change feeds, and separate cursors for user data and catalog data.

```
Device
|
|-- sends sync_mutations
|
|-- pulls user_sync_change_log after last user cursor
|
|-- pulls catalog_sync_change_log per domain after last catalog cursor
```

devices

Purpose: Registered application installations or client devices.

Main data: user_id, device_name, platform, app_version, last_seen_at, registered_at, revoked_at.

Key relationships: Belongs to one profile and owns sync-state rows.

Typical API surface

Endpoint	Purpose
POST /devices/register	Register device
GET /me/devices	List user's devices
POST /devices/{id}/revoke	Revoke a lost or retired device

device_user_sync_state

Purpose: Tracks how far one device has consumed the user's personal change feed.

Main data: device_id, last_user_cursor, last_pull_at, last_push_at, full_resync_required.

Key relationships: One-to-one with a device.

Typical API surface

Endpoint	Purpose
GET /sync/user?after={cursor}	Pull personal changes after cursor

Design note: The client updates its durable local cursor only after successfully applying the returned batch.

device_catalog_sync_state

Purpose: Tracks catalog synchronization state per device and domain.

Main data: device_id, domain_id, last_catalog_cursor, last_catalog_release_id, full_resync_required.

Key relationships: Composite key supports independent synchronization of multiple domains.

Typical API surface

Endpoint	Purpose
GET /sync/catalog/{domainId}?after={cursor}	Pull catalog changes for one domain

sync_mutations

Purpose: Device-to-server mutation inbox and idempotency ledger.

Main data: mutation_id, user_id, device_id, entity type/id, operation, payload, request_hash, status, row versions, errors, retention timestamps.

Key relationships: Composite FK guarantees device ownership by the same user.

Typical API surface

Endpoint	Purpose
POST /sync/mutations	Submit one mutation
POST /sync/mutations/batch	Optional batch mutation endpoint
GET /sync/mutations/{mutationId}	Optional status lookup

Design note: The same mutation_id must not represent two different request bodies. request_hash supports that validation.

user_sync_counters

Purpose: Transactional cursor allocator state per user.

Main data: user_id, last_cursor, updated_at.

Key relationships: One counter row per user.

Design note: Internal only. Do not expose direct client writes.

user_sync_change_log

Purpose: Ordered server-to-device delta feed for user-owned state.

Main data: user_id, cursor, entity type/id, operation, entity_version, changed_at, retain_until.

Key relationships: Primary key user_id + cursor supports ordered cursor reads.

Example feed:
141 bookmark upsert
142 note update
143 progress update

Typical API surface

Endpoint	Purpose
GET /sync/user?after={cursor}	Return changes and next cursor

catalog_sync_counters

Purpose: Transactional cursor allocator state per content domain.

Main data: domain_id, last_cursor, updated_at.

Key relationships: One counter row per domain.

Design note: Internal only.

catalog_sync_change_log

Purpose: Ordered delta feed for catalog changes within one domain.

Main data: domain_id, cursor, entity type/id, operation, entity_version, changed_at, retain_until.

Key relationships: Primary key domain_id + cursor supports independent domain feeds.

Typical API surface

Endpoint	Purpose
GET /sync/catalog/{domainId}?after={cursor}	Return catalog delta

10. Supabase-Managed Schemas

The schemas auth, storage, realtime, and vault are Supabase infrastructure. They are not part of the 34-table RecallStack domain model.

auth.*

Supabase authentication internals: users, sessions, identities, refresh tokens, MFA, OAuth and SSO data.

Application use: Use Supabase Auth APIs. The application-level relationship is auth.users.id -> public.profiles.id.

storage.*

Metadata for Supabase Storage buckets and objects.

Application use: Use for avatars, diagrams, images, attachments and other binary assets. Store only references/paths in application tables.

realtime.*

Supabase Realtime internal metadata and subscription plumbing.

Application use: Use through Supabase Realtime features rather than building CRUD APIs on these tables.

vault.secrets

Secret storage.

Application use: Use for integration secrets or service credentials when appropriate; do not expose as normal application CRUD.

11. Recommended API Structure

Do not create one controller per table. Organize the API around business capabilities and commands.

/api/v1

/profile

/domains

/categories

/topics

/content

/content/{slug}

/me/progress

/me/bookmarks

/me/notes

/practice/resources

/practice/attempts

/reviews/due

/reviews/{cardId}/submit

/devices

/sync/mutations

/sync/user

/sync/catalog

/admin/content

/admin/content/{id}/versions

/admin/content-versions/{id}/publish

/admin/catalog/releases

/admin/users/{id}/roles

Business-action APIs, not table CRUD

Avoid	Prefer
POST /review-history + PATCH /review-card	POST /reviews/{cardId}/submit
PATCH /content-version + PATCH /content-item	POST /content-versions/{id}/publish
POST /topic-categories	PUT /admin/topics/{id}/categories
POST /content-version-blocks repeatedly	PUT /admin/content-versions/{id}/document

The API operation should match the transaction boundary. This keeps business invariants in one place and reduces inconsistent intermediate states.

12. Critical Transaction Boundaries

Publish content version

Validate ownership -> validate state transition -> update version -> insert status history -> update current_published_version_id -> emit catalog change -> commit.

Submit review

Deduplicate review_event_id -> lock/CAS review card -> calculate next schedule -> insert review_history -> update review_card -> emit user change -> commit.

Apply sync mutation

Claim mutation_id -> verify request_hash -> validate device ownership -> validate base row version -> update authoritative entity -> allocate cursor -> write change log -> mark mutation applied -> commit.

Publish catalog release

Validate every item/version/domain mapping -> freeze manifest -> set published status/time -> emit catalog changes if needed -> commit.

Grant or revoke role

Validate actor permission -> insert grant or set revoked_at -> preserve audit fields -> commit.

13. Quick Reference: All 34 RecallStack Tables

Table	Area	Responsibility
profiles	Identity	Application profile for auth.users user
roles	Identity	Role vocabulary
profile_role_grants	Identity	Role grant/revocation history
domains	Taxonomy	Top-level learning subject
categories	Taxonomy	Hierarchical navigation
topics	Taxonomy	Concepts and patterns
topic_categories	Taxonomy	Topic-to-category mapping
content_items	Content	Stable learning-object identity
content_versions	Content	Editorial revisions
content_version_status_history	Content	Workflow audit history
content_blocks	Content	Immutable structured content blocks
content_version_blocks	Content	Ordered version composition
content_item_categories	Content	Content-category mapping
content_item_topics	Content	Content-topic mapping
content_relations	Content	Prerequisite/related/alternative graph
practice_providers	Practice	Practice source registry
practice_resources	Practice	Concrete practice links/questions
practice_attempts	Practice	Append-only user attempts
user_progress	Learning	Current status and confidence
bookmarks	Learning	Saved-content set
user_notes	Learning	Private editable notes
activity_events	Learning	Observational event stream
review_cards	Recall	Current scheduling state
review_history	Recall	Immutable review event history
catalog_releases	Catalog	Domain snapshot identity
catalog_release_versions	Catalog	Release manifest
devices	Sync	Registered user devices
device_user_sync_state	Sync	Per-device user-feed cursor
device_catalog_sync_state	Sync	Per-device, per-domain catalog cursor
sync_mutations	Sync	Inbound mutation idempotency ledger
user_sync_counters	Sync	Per-user cursor allocator state
user_sync_change_log	Sync	Ordered user-data change feed
catalog_sync_counters	Sync	Per-domain cursor allocator state
catalog_sync_change_log	Sync	Ordered catalog change feed